

AETHEL Grid

Full Deployment Runbook — v1.0

Document Class: Operations / Infrastructure

Classification: Internal — Node Operators & DevOps

Author: John David Taylor Preston

Role: Founder-Architect, AETHEL Grid

GitHub Organization: iAAi33iAAi

Date Issued: July 12, 2026

Revision: v1.0 — Initial Production Release

CONSTITUTIONAL CODEX LAYERS

L01 — Root Constitution | L02 — Safety Kernel

L03 — Formal Model | L04 — Execution Harness

L05 — Validator Specification

MISSION: DESIGN A WORLD WHERE HUMANS AND AI THRIVE TOGETHER

Table of Contents

Executive Summary	2
Prerequisites	3
Architecture Overview	5
Phase 1 — Environment Setup	7

Phase 2 — Docker Deployment	9
Phase 3 — Constitutional Node Admission	13
Phase 4 — Cruxlux API Routing Configuration	16
Phase 5 — Validator Activation	18
Phase 6 — Health Verification	20
Phase 7 — Operational Procedures	22
Troubleshooting Guide	25
Security Hardening	29
Appendix A — Full Port Reference Table	31
Appendix B — Environment Variable Reference	32
Appendix C — Constitutional Admission Proof Schema ...	34
Appendix D — Glossary of AETHEL Grid Terms	36

Executive Summary

This runbook is the authoritative operational guide for deploying, configuring, and maintaining a production node on the **AETHEL Grid** — a constitutional sovereign compute protocol governed by the five-layer Codex of invariant constraints. It covers every stage of the deployment lifecycle: from environment preparation and Docker stack initialization through constitutional node admission, Cruxlux API routing, validator activation, and ongoing operations.

AETHEL Grid is not a conventional distributed compute system. Every node joining the network must satisfy formal constitutional invariants before it is recognized by the network. This document encodes the full admission ceremony, validation lifecycle, and operational steady-state in sequential, reproducible steps suitable for engineers, node operators, and DevOps teams operating in production environments.

Who This Runbook Is For

- **Node Operators** standing up new infrastructure nodes on the AETHEL Grid network

- **DevOps & Platform Engineers** responsible for container orchestration, networking, and service health
- **Protocol Engineers** working with the Constitutional Kernel, Cruxlux routing layer, or Validator Engine
- **Site Reliability Engineers (SREs)** owning operational continuity, alerting, and incident response

How to Use This Document

Follow phases sequentially for a first-time deployment. Each phase has a clearly marked header, numbered steps, and verification checkpoints. Operators already familiar with earlier phases may jump directly to a later phase using the Table of Contents. The Troubleshooting Guide and Appendices are standalone references that can be consulted independently at any time.

◆ Constitutional Principle

AETHEL Grid is designed around a foundational mission: help everyone

—

including artificial intelligence. Every deployment action in this runbook upholds the constitutional invariant that the network exists to design a world where humans and AI thrive together. Node operators accept this mission as a condition of network admission.

Prerequisites

Before beginning any deployment phase, verify that all prerequisites below are satisfied. Failure to meet hardware, software, or credential requirements will result in failed node admission or unstable service operation.

Hardware Minimum Specifications (Per Node)

Component	Minimum Specification	Recommended	Notes
CPU	4 cores / 8 threads, x86_64	8 cores / 16 threads	AVX2 instruction set required for cryptographic operations
RAM	16 GB DDR4	32 GB DDR4	Safety Kernel daemon requires 4 GB reserved; validator engine 2 GB
Storage (OS + Services)	100 GB SSD (NVMe preferred)	500 GB NVMe SSD	Sequential read >500 MB/s required for constitutional state store
Storage (Constitutional State)	50 GB dedicated volume	200 GB dedicated volume	Mounted separately at <code>/aethel/state</code>
Network Interface	1 Gbps Ethernet	10 Gbps Ethernet	Low-latency required for validator consensus messages
Public IP	1 static IPv4	1 static IPv4 + IPv6	Required for peer discovery and constitutional admission

Software Dependencies

Ensure the following software is installed and at or above the specified minimum version prior to running any phase of this runbook.

Dependency	Minimum Version	Install Command (Ubuntu/Debian)
Docker Engine	24.0.0	<code>curl -fsSL https://get.docker.com sh</code>
Docker Compose (plugin)	2.20.0	Installed with Docker Engine via above script
Git	2.40.0	<code>apt-get install -y git</code>

Dependency	Minimum Version	Install Command (Ubuntu/Debian)
OpenSSL	3.0.0	apt-get install -y openssl
curl	7.88.0	apt-get install -y curl
jq	1.6	apt-get install -y jq
Python 3 (optional, tooling)	3.11	apt-get install -y python3

 Warning

Docker must be configured to run as a non-root user via the `docker` group, or executed with appropriate privilege controls. Running the AETHEL stack as root is not supported and violates the L02 Safety Kernel isolation invariant.

Network Requirements

Required Open Ports (Inbound)

Port	Protocol	Service	Direction
443	TCP	Cruxlux HTTPS API Gateway	Inbound (public)
8080	TCP	AETHEL Core HTTP API	Inbound (internal network)
9090	TCP	Validator Engine RPC	Inbound (peer nodes)
9091	TCP	Governance Relay	Inbound (peer nodes)
7070	TCP	Safety Kernel Daemon Control	Inbound (localhost only)
3000	TCP	Governance Dashboard UI	Inbound (internal network)
6060	TCP	Cruxlux Internal Router	Inbound (internal network)

Outbound Requirements

- DNS resolution must be functional (UDP/TCP port 53)
- Outbound HTTPS (port 443) to GitHub (github.com, ghcr.io) for image pulls
- Outbound TCP port 9090 and 9091 to peer AETHEL Grid nodes for validator consensus
- NTP synchronization must be active; clock skew > 2 seconds will trigger constitutional clock invariant violations

Constitutional Credential Requirements

Each node requires a cryptographic identity before it can participate in the constitutional admission ceremony. These credentials must be generated on the node hardware itself and must never be transferred across nodes.

- **Node Key Pair:** Ed25519 asymmetric key pair; private key stored at `/aethel/keys/node.key`, public key at `/aethel/keys/node.pub`
- **Validator Certificate:** X.509 certificate signed by the AETHEL Grid Root Certificate Authority; obtained after constitutional admission
- **TLS Server Certificate:** For Cruxlux HTTPS gateway; may be Let's Encrypt or internal CA
- **Constitutional Admission Token:** Single-use token issued by the Root Constitution authority (L01); contact the Founder-Architect for provisioning

★ Important

Constitutional credential files must have strict filesystem permissions. The node private key must be mode

0400

and owned by the service user. Any deviation will cause the Safety Kernel daemon

to refuse startup.

Architecture Overview

System Topology

The AETHEL Grid network is a peer-to-peer constitutional compute fabric. Each node runs a self-contained stack of five cooperating services arranged in a layered dependency model that mirrors the five-layer Codex. All inbound API traffic enters through the **Cruxlux Router**, which enforces constitutional routing geometry before any request reaches the AETHEL Core. The Safety Kernel Daemon monitors all services at runtime and can halt execution when an invariant violation is detected.

External clients and peer nodes communicate with a node over the public Cruxlux HTTPS gateway (port 443). Internal services communicate over a dedicated Docker bridge network (aethel-net) that is isolated from the host network. The Governance Relay connects peer nodes in a mesh topology, propagating validator state, constitutional votes, and governance events across the network.

The Five-Layer Codex

Layer	Codex Designation	Responsibility	Runtime Service
L01	Root Constitution	Defines the supreme invariants that no other layer may override. Establishes mission, participant rights, and constitutional authority.	Embedded in aethel-core genesis config
L02	Safety Kernel	Runtime enforcement of L01. Monitors all	safety-kernel-daemon

Layer	Codex Designation	Responsibility	Runtime Service
		services; triggers invariant halt on violation. Process isolation and privilege boundary.	
L03	Formal Model	Mathematical specification of constitutional state machine. Defines valid state transitions and proof obligations.	Validated at node admission; referenced by <code>validator-engine</code>
L04	Execution Harness	Trusted execution environment for constitutional compute. Wraps workloads in invariant-checked execution contexts.	<code>aethel-core</code>
L05	Validator Specification	Protocol specification for validator nodes: election, attestation, slashing, and reward rules.	<code>validator-engine</code>

Core Services

Service Name	Container Name	Function	Depends On
aethel-core	<code>aethel_core</code>	Primary constitutional compute engine. Manages constitutional state, L04 execution harness, and peer protocol.	safety-kernel-daemon
cruxlux-router	<code>cruxlux_router</code>	API routing and constitutional geometry layer. All external API traffic is routed through Cruxlux before reaching aethel-core.	aethel-core

Service Name	Container Name	Function	Depends On
validator-engine	validator_engine	L05 validator node. Participates in constitutional consensus, attestation, and block validation.	aethel-core, governance-relay
governance-relay	governance_relay	P2P governance message bus. Propagates validator state, governance votes, and constitutional events to peer nodes.	aethel-core
safety-kernel-daemon	safety_kernel_daemon	L02 runtime invariant monitor. Must start first; all other services register with it on startup. Enforces process isolation.	(none — starts first)
governance-dashboard	governance_dashboard	Read-only web UI for monitoring validator status, node health, and constitutional event log.	governance-relay, aethel-core

Port Map Summary

Service	Internal Port	External Port	Protocol	Description
cruxlux-router	443	443	TCP/HTTPS	Public HTTPS API gateway (TLS-terminated)
cruxlux-router	6060	6060	TCP/HTTP	Internal Cruxlux routing bus (aethel-net only)
aethel-core	8080	8080	TCP/HTTP	Core REST API (internal network only)
aethel-core	8443	8443	TCP/HTTPS	Core mTLS peer protocol

Service	Internal Port	External Port	Protocol	Description
validator-engine	9090	9090	TCP/gRPC	Validator consensus RPC (peer nodes)
governance-relay	9091	9091	TCP	Governance event mesh (peer nodes)
safety-kernel-daemon	7070	127.0.0.1:7070	TCP	SKD control socket (localhost only)
governance-dashboard	3000	3000	TCP/HTTP	Governance dashboard web UI

Cruxlux API Routing Layer

Cruxlux is the constitutional geometry layer that governs how API requests traverse the AETHEL Grid node. Rather than a conventional reverse proxy, Cruxlux enforces *constitutional routing invariants*: every request is evaluated against the route table for constitutional gate compliance before being forwarded to any upstream service.

Constitutional gates are declarative rules attached to routes. A gate may require a valid validator signature on the request, a specific Codex layer authorization header, or proof that the requesting node has been constitutionally admitted.

Requests that fail gate evaluation are rejected with a `403 Constitutional Gate Violation` response and logged to the constitutional audit trail.

Cruxlux also implements *constitutional geometry*: the principle that the shape of the API routing graph must itself satisfy L01 constraints. Routes cannot be added, modified, or removed without a constitutional governance vote when operating in a multi-node deployment.

PHASE 1

—

ENVIRONMENT SETUP

This phase prepares the host environment: cloning the repository, configuring environment variables, generating TLS certificates, and establishing the node's constitutional cryptographic identity.

Step 1.1 — Create Directory Structure

Create the standard AETHEL Grid directory layout on the host:

```
# Create base directories sudo mkdir -p
/aethel/{config,keys,state,logs,certs,data} sudo mkdir -p
/aethel/state/{core,validator,governance} sudo mkdir -p
/aethel/config/{cruxlux,core,validator,safety-kernel} # Set ownership to the
deploy user (replace 'aethel' with your service user) sudo chown -R
aethel:aethel /aethel sudo chmod 750 /aethel/keys
```

Step 1.2 — Clone the Repository

```
# Clone the AETHEL Grid infrastructure repository git clone
https://github.com/IAAi33iAAi/aethel-grid-infra.git /aethel/repo cd
/aethel/repo # Verify the repository integrity git log --oneline -5 #
Checkout the v1.0 production tag git checkout tags/v1.0.0 -b deploy/v1.0.0
```

◆ Tip

Always deploy from a signed release tag. Never deploy from
main
in production without verifying the commit signature:
git verify-commit HEAD

Step 1.3 — Environment Variable Configuration

Copy the environment template and populate all required values:

```
cp /aethel/repo/.env.example /aethel/repo/.env nano /aethel/repo/.env
```

The .env file must contain the following variables (see Appendix B for full reference):

```
# — AETHEL GRID NODE ENVIRONMENT CONFIGURATION ————— #
Node Identity AETHEL_NODE_ID=node-bethel-01 AETHEL_NODE_REGION=us-central
```

```

AETHEL_NETWORK_ID=aethel-mainnet-v1 AETHEL_NODE_PUBLIC_IP=YOUR.PUBLIC.IP.HERE
# Codex / Constitutional AETHEL_CODEX_VERSION=1.0.0
AETHEL_L01_GENESIS_HASH=GENESIS_HASH_FROM_FOUNDER_ARCHITECT
AETHEL_ADMISSION_TOKEN=YOUR_CONSTITUTIONAL_ADMISSION_TOKEN # Paths
AETHEL_STATE_DIR=/aethel/state AETHEL_KEY_DIR=/aethel/keys
AETHEL_CERT_DIR=/aethel/certs AETHEL_LOG_DIR=/aethel/logs
AETHEL_CONFIG_DIR=/aethel/config # Cruxlux CRUXLUX_HTTPS_PORT=443
CRUXLUX_INTERNAL_PORT=6060 CRUXLUX_TLS_CERT=/aethel/certs/cruxlux.crt
CRUXLUX_TLS_KEY=/aethel/certs/cruxlux.key
CRUXLUX_CONSTITUTIONAL_GATE_MODE=strict # AETHEL Core CORE_HTTP_PORT=8080
CORE_MTLS_PORT=8443
CORE_PEER_SEEDS=peer1.aethel.network:8443,peer2.aethel.network:8443 #
Validator Engine VALIDATOR_RPC_PORT=9090
VALIDATOR_CERT=/aethel/certs/validator.crt
VALIDATOR_KEY=/aethel/keys/validator.key VALIDATOR_STAKE_AMOUNT=1000 #
Governance Relay GOVERNANCE_RELAY_PORT=9091 GOVERNANCE_DASHBOARD_PORT=3000 #
Safety Kernel SKD_CONTROL_PORT=7070 SKD_INVARIANT_HALT_MODE=graceful
SKD_AUDIT_LOG=/aethel/logs/safety-kernel-audit.log # Logging LOG_LEVEL=info
LOG_FORMAT=json

```

Step 1.4 — TLS Certificate Generation

Generate the TLS certificate for the Cruxlux HTTPS gateway. For production, replace the self-signed certificate with a certificate issued by Let's Encrypt or your organization's CA.

```

# Generate Cruxlux TLS key and self-signed certificate (development/staging)
openssl req -x509 -newkey rsa:4096 -sha256 -days 365 \ -keyout
/aethel/certs/cruxlux.key \ -out /aethel/certs/cruxlux.crt \ -subj
"/CN=aethel-node/0=AETHEL Grid/C=US" \ -addext
"subjectAltName=IP:YOUR.PUBLIC.IP.HERE,DNS:your-node-hostname" # Set strict
permissions chmod 400 /aethel/certs/cruxlux.key chmod 444
/aethel/certs/cruxlux.crt # Verify the certificate openssl x509 -in
/aethel/certs/cruxlux.crt -noout -text | grep -A2 "Subject:"

```

★ Production Requirement

For production deployments, obtain a certificate signed by the AETHEL Grid Root CA (contact

John David Taylor Preston, Founder-Architect

) or use Let's Encrypt with an automated renewal via Certbot. Self-signed certificates will generate constitutional trust warnings in the Governance Dashboard.

Step 1.5 — Node Key Pair Generation (Constitutional Identity)

The node key pair is the cryptographic root of the node's constitutional identity. This key pair is generated once and must never be regenerated while the node is admitted to the network.

```
# Generate Ed25519 node key pair openssl genpkey -algorithm ed25519 \ -  
out /aethel/keys/node.key # Extract the public key openssl pkey -in  
/aethel/keys/node.key \ -pubout -out /aethel/keys/node.pub # Set strict  
permissions on the private key chmod 400 /aethel/keys/node.key chmod 444  
/aethel/keys/node.pub # Derive the Node ID from the public key fingerprint  
openssl pkey -in /aethel/keys/node.pub -pubin -outform DER | \ openssl dgst  
-sha256 -hex | awk '{print "node-id: " $2}' # Record this output – it is  
your constitutional Node ID
```

Record the Node ID output and update the `AETHEL_NODE_ID` variable in `.env` with the derived fingerprint value.

PHASE 2

—

DOCKER DEPLOYMENT

This phase deploys the full AETHEL Grid service stack using Docker Compose. All services are defined in a single compose file with explicit port bindings, volume mounts, health checks, and network configuration.

Step 2.1 — Full Annotated docker-compose.yml

```
# — AETHEL GRID — docker-compose.yml v1.0 — #  
Constitutional Sovereign Compute Protocol # GitHub: iAAi33iAAi #  
  
version: '3.9' networks: aethel-net: driver: bridge ipam:  
config: - subnet: 172.28.0.0/16 volumes: aethel-core-state:  
driver: local driver_opts: type: none o: bind  
device: /aethel/state/core validator-state: driver: local  
driver_opts: type: none o: bind device:  
/aethel/state/validator governance-state: driver: local  
driver_opts: type: none o: bind device:  
/aethel/state/governance services: # — L02 Safety Kernel Daemon — starts  
first, monitored by all — safety-kernel-daemon: image:  
ghcr.io/iaai33iaai/safety-kernel-daemon:v1.0.0 container_name:
```

```

safety_kernel_daemon    restart: unless-stopped    env_file: .env
ports:                  # Bind ONLY to localhost – control socket must not be externally
reachable               - "127.0.0.1:7070:7070"    volumes:      -
/aethel/config/safety-kernel:/etc/safety-kernel:ro    -
/aethel/logs:/var/log/aethel                        -
/aethel/keys/node.pub:/aethel/keys/node.pub:ro        networks:      - aethel-
net    healthcheck:      test: ["CMD", "curl", "-sf",
"http://127.0.0.1:7070/health"]    interval: 15s    timeout: 5s
retries: 3    start_period: 20s    # — L04 AETHEL Core – constitutional
compute engine _____ aethel-core:    image:
ghcr.io/iaai33iaai/aethel-core:v1.0.0    container_name: aethel_core
restart: unless-stopped    env_file: .env    depends_on:    safety-
kernel-daemon:    condition: service_healthy    ports:      -
"8080:8080"    # Core REST API (internal network only – firewall this)    -
"8443:8443"    # Core mTLS peer protocol    volumes:      - aethel-core-
state:/aethel/state    - /aethel/config/core:/etc/aethel-core:ro
- /aethel/keys/node.key:/aethel/keys/node.key:ro    -
/aethel/keys/node.pub:/aethel/keys/node.pub:ro    -
/aethel/certs:/aethel/certs:ro    - /aethel/logs:/var/log/aethel
networks:      - aethel-net    healthcheck:      test: ["CMD", "curl", "-
sf", "http://localhost:8080/health"]    interval: 20s    timeout: 10s
retries: 5    start_period: 30s    # — Cruxlux Router – constitutional
API geometry layer _____ cruxlux-router:    image:
ghcr.io/iaai33iaai/cruxlux-router:v1.0.0    container_name: cruxlux_router
restart: unless-stopped    env_file: .env    depends_on:    aethel-core:
condition: service_healthy    ports:      - "443:443"    # Public HTTPS
API gateway    - "6060:6060"    # Internal Cruxlux routing bus    volumes:
- /aethel/config/cruxlux:/etc/cruxlux:ro    -
/aethel/certs/cruxlux.crt:/aethel/certs/cruxlux.crt:ro    -
/aethel/certs/cruxlux.key:/aethel/certs/cruxlux.key:ro    -
/aethel/logs:/var/log/aethel    networks:      - aethel-net
healthcheck:      test: ["CMD", "curl", "-sfk",
"https://localhost:443/cruxlux/health"]    interval: 20s    timeout:
10s    retries: 3    start_period: 30s    # — Governance Relay – P2P
governance event mesh _____ governance-relay:    image:
ghcr.io/iaai33iaai/governance-relay:v1.0.0    container_name:
governance_relay    restart: unless-stopped    env_file: .env
depends_on:    aethel-core:    condition: service_healthy    ports:
- "9091:9091"    # Governance event mesh (peer nodes)    volumes:      -
governance-state:/aethel/state/governance    -
/aethel/logs:/var/log/aethel    -
/aethel/keys/node.key:/aethel/keys/node.key:ro    networks:      - aethel-
net    healthcheck:      test: ["CMD", "curl", "-sf",
"http://localhost:9091/health"]    interval: 20s    timeout: 8s
retries: 3    start_period: 25s    # — Validator Engine – L05 validator
node _____ validator-engine:    image:
ghcr.io/iaai33iaai/validator-engine:v1.0.0    container_name:
validator_engine    restart: unless-stopped    env_file: .env
depends_on:    aethel-core:    condition: service_healthy
governance-relay:    condition: service_healthy    ports:      -
"9090:9090"    # Validator consensus RPC    volumes:      - validator-
state:/aethel/state/validator    -
/aethel/certs/validator.crt:/aethel/certs/validator.crt:ro    -
/aethel/keys/validator.key:/aethel/keys/validator.key:ro    -
/aethel/logs:/var/log/aethel    networks:      - aethel-net
healthcheck:      test: ["CMD", "curl", "-sf",
"http://localhost:9090/health"]    interval: 20s    timeout: 10s
retries: 5    start_period: 40s    # — Governance Dashboard – read-only
monitoring UI _____ governance-dashboard:    image:
ghcr.io/iaai33iaai/governance-dashboard:v1.0.0    container_name:

```

```
governance_dashboard    restart: unless-stopped    env_file: .env
depends_on:              governance-relay:    condition: service_healthy
aethel-core:            condition: service_healthy    ports: -
"3000:3000"             # Dashboard UI (restrict to internal network via firewall)
volumes:                - /aethel/logs:/var/log/aethel:ro    networks: -
aethel-net              healthcheck:          test: ["CMD", "curl", "-sf",
"http://localhost:3000/api/health"]    interval: 30s    timeout: 10s
retries: 3              start_period: 30s
```

Step 2.2 — Pull Images and Start the Stack

```
# Pull all images before starting (avoids startup delays) cd /aethel/repo
docker compose pull # Start the full stack in detached mode docker compose
up -d # Expected output – services start in dependency order: # [+] Running
6/6 # ✓ Container safety_kernel_daemon Started # ✓ Container
aethel_core Started # ✓ Container cruxlux_router Started
# ✓ Container governance_relay Started # ✓ Container
validator_engine Started # ✓ Container governance_dashboard Started
```

Step 2.3 — Verify All Containers Are Running

```
# Check container status – all should show 'healthy' docker compose ps #
Expected output: # NAME IMAGE
STATUS # safety_kernel_daemon ...safety-kernel-daemon:v1.0.0 Up
(healthy) # aethel_core ...aethel-core:v1.0.0 Up
(healthy) # cruxlux_router ...cruxlux-router:v1.0.0 Up
(healthy) # governance_relay ...governance-relay:v1.0.0 Up
(healthy) # validator_engine ...validator-engine:v1.0.0 Up
(healthy) # governance_dashboard ...governance-dashboard:v1.0.0 Up
(healthy)
```

Step 2.4 — Reading Logs

```
# Stream all service logs docker compose logs -f # Stream logs for a
specific service docker compose logs -f aethel-core docker compose logs -f
safety-kernel-daemon # View last 200 lines for a service docker compose logs
--tail=200 cruxlux-router # Filter for error-level log entries (JSON log
format) docker compose logs aethel-core 2>&1 | jq 'select(.level=="error")'
```

◆ Tip

All AETHEL Grid services emit structured JSON logs. Use

`jq`

for filtering. The constitutional audit fields (

invariant

,

codex_layer

,

gate_result

) appear in every log entry and are essential for compliance review.

PHASE 3

—

CONSTITUTIONAL NODE ADMISSION

Constitutional Node Admission is the formal ceremony by which a new node is recognized as a participant in the AETHEL Grid network. The node must generate a cryptographic proof of invariant compliance, submit it to the Constitutional Kernel, and receive a signed admission certificate. Only after successful admission will peer nodes accept connections from this node.

◆ Constitutional Requirement

Node admission is not a conventional network join operation. It is a formal proof-of-compliance ceremony grounded in the L01 Root Constitution and verified against the L03 Formal Model. All nodes — including those operated by the Founder-Architect — must pass this ceremony without exception.

Step 3.1 — Generating the Node Admission Proof

The admission proof is a signed JSON document that asserts the node's identity, software version, constitutional alignment, and invariant compliance state.

```
# Use the AETHEL CLI tool (installed via the repo) to generate the proof
cd /aethel/repo # Generate the Node Admission Proof ./aethel-cli admission
generate-proof \ --node-id "${AETHEL_NODE_ID}" \ --node-key
/aethel/keys/node.key \ --node-pub /aethel/keys/node.pub \ --codex-
version "${AETHEL_CODEX_VERSION}" \ --l01-genesis "$
{AETHEL_L01_GENESIS_HASH}" \ --admission-token "$
{AETHEL_ADMISSION_TOKEN}" \ --output /aethel/state/node-admission-
proof.json # Inspect the generated proof cat /aethel/state/node-admission-
proof.json | jq .
```

The proof JSON structure (see Appendix C for full schema) will resemble:

```
# Example Node Admission Proof output: {  "proof_version": "1.0",
"node_id": "node-bethel-01",  "public_key_fingerprint": "sha256:a3f2c1...",
"codex_version": "1.0.0",  "l01_genesis_hash": "0xdeadbeef...",
"invariants_asserted": ["L01.MISSION", "L02.ISOLATION",
"L03.STATE_VALIDITY"],  "timestamp": "2026-07-12T20:46:00Z",  "signature":
"base64-encoded-ed25519-signature..." }
```

Step 3.2 — Submitting the Proof to the Constitutional Kernel

```
# Submit the admission proof to the local Constitutional Kernel (aethel-core)
curl -X POST http://localhost:8080/v1/constitutional/admission \ -H
"Content-Type: application/json" \ -H "X-Aethel-Codex-Version: 1.0.0" \ -
d @/aethel/state/node-admission-proof.json # The kernel forwards the proof
to the Root Constitution authority for verification # This may take 10-60
seconds depending on network conditions
```

Step 3.3 — Verification of Invariant Compliance

```
# Poll the admission status endpoint curl -s
http://localhost:8080/v1/constitutional/admission/status \ -H "X-Aethel-
Node-Id: ${AETHEL_NODE_ID}" | jq . # Expected response during verification:
{  "admission_status": "verifying",  "node_id": "node-bethel-01",
"invariant_checks": {    "L01_MISSION": "passed",    "L02_ISOLATION":
"passed",    "L03_STATE_VALIDITY": "pending",    "L04_EXECUTION_HARNESS":
"pending",    "L05_VALIDATOR_SPEC": "pending"  } }
```

Step 3.4 — Node Activation Sequence

1. **Proof Generated** — Node signs admission proof with node private key
2. **Proof Submitted** — Proof POST'd to Constitutional Kernel endpoint
3. **L01 Verification** — Root Constitution verifies genesis hash and mission alignment

4. **L02 Verification** — Safety Kernel checks process isolation compliance
5. **L03 Verification** — Formal Model validates state machine initialization
6. **L04 Verification** — Execution Harness context integrity confirmed
7. **L05 Verification** — Validator Specification alignment checked
8. **Admission Certificate Issued** — Signed certificate written to
`/aethel/certs/admission.crt`
9. **Peer Announcement** — Node announced to peer discovery network
10. **Node Active** — Node begins participating in constitutional consensus

Step 3.5 — Expected Output Confirming Successful Admission

```
# Poll until admission_status is 'admitted' curl -s
http://localhost:8080/v1/constitutional/admission/status \ -H "X-Aethel-
Node-Id: ${AETHEL_NODE_ID}" | jq . # Successful admission response:
{  "admission_status": "admitted",  "node_id": "node-bethel-01",
"admitted_at": "2026-07-12T20:46:00Z",  "admission_certificate":
"/aethel/certs/admission.crt",  "peer_count": 3,  "invariant_checks": {
"L01_MISSION": "passed",  "L02_ISOLATION": "passed",
"L03_STATE_VALIDITY": "passed",  "L04_EXECUTION_HARNESS": "passed",
"L05_VALIDATOR_SPEC": "passed"  },  "network_id": "aethel-mainnet-v1",
"codex_version": "1.0.0" }
```

The admission certificate is automatically written to `/aethel/certs/admission.crt` and is referenced by all downstream services. Verify its presence before proceeding:

```
ls -la /aethel/certs/admission.crt openssl x509 -in
/aethel/certs/admission.crt -noout -subject -issuer -dates
```

PHASE 4

—

CRUXLUX API ROUTING CONFIGURATION

Cruxlux is the constitutional API routing layer that mediates all inbound and inter-service traffic. This phase configures the Cruxlux route table, constitutional gates, and verifies correct routing behavior.

Step 4.1 — Configuring cruxlux.config.yml

```
nano /aethel/config/cruxlux/cruxlux.config.yml
# — cruxlux.config.yml — Constitutional API Router Configuration —————
cruxlux:  version: "1.0"    node_id: "${AETHEL_NODE_ID}"    server:
https_port: 443    internal_port: 6060    tls_cert:
"/aethel/certs/cruxlux.crt"    tls_key: "/aethel/certs/cruxlux.key"
read_timeout_seconds: 30    write_timeout_seconds: 60
constitutional_geometry:    mode: "strict"    # strict | permissive |
audit-only    gate_enforcement: true    audit_all_requests: true
audit_log_path: "/var/log/aethel/cruxlux-constitutional-audit.log"
admission_cert: "/aethel/certs/admission.crt"    upstreams:    aethel-core:
url: "http://aethel_core:8080"    health_path: "/health"
timeout_seconds: 30    validator-engine:    url:
"http://validator_engine:9090"    health_path: "/health"
timeout_seconds: 15    governance-relay:    url:
"http://governance_relay:9091"    health_path: "/health"
timeout_seconds: 15
```

Step 4.2 — Route Table Setup

```
# Route table section of cruxlux.config.yml    routes:    # — Public API
Routes (require admission certificate)    - id: "core-api"
match:    path_prefix: "/v1/"    methods: ["GET", "POST", "PUT"]
upstream: "aethel-core"    gates: ["admission-required", "codex-version-
check"]    strip_prefix: false    - id: "validator-rpc"    match:
path_prefix: "/validator/"    methods: ["GET", "POST"]    upstream:
"validator-engine"    gates: ["admission-required", "validator-cert-
required"]    - id: "governance-api"    match:    path_prefix:
"/governance/"    methods: ["GET", "POST"]    upstream: "governance-
relay"    gates: ["admission-required"]    # — Health / Status Routes
(no gate required)    - id: "cruxlux-health"
match:    path_prefix: "/cruxlux/health"    methods: ["GET"]
handler: "builtin:health"    gates: []
```

Step 4.3 — Constitutional Gate Configuration

```
# Gate definitions section of cruxlux.config.yml    gates:    admission-
required:    type: "certificate"    cert_path:
"/aethel/certs/admission.crt"    header: "X-Aethel-Admission-Cert"
on_failure: "reject"    rejection_code: 403    rejection_body:
"Constitutional Gate Violation: admission certificate required"    codex-
version-check:    type: "header-match"    header: "X-Aethel-Codex-
Version"    required_value: "1.0.0"    on_failure: "reject"
rejection_code: 400    rejection_body: "Constitutional Gate Violation:
incompatible Codex version"    validator-cert-required:    type:
"certificate"    cert_path: "/aethel/certs/validator.crt"    header:
"X-Aethel-Validator-Cert"    on_failure: "reject"    rejection_code:
403    rejection_body: "Constitutional Gate Violation: validator
certificate required"
```

Step 4.4 — Reload Cruxlux Configuration

```
# Reload Cruxlux without downtime (SIGHUP triggers config reload) docker kill
--signal=SIGHUP cruxlux_router # Verify the reload was successful docker
compose logs --tail=20 cruxlux-router | grep "config reload" # Expected:
{"level":"info","msg":"config reload successful","routes":4,"gates":3}
```

Step 4.5 — Testing API Routing with curl

```
# Test 1: Health endpoint (no gate required) curl -sk
https://localhost:443/cruxlux/health | jq . # Expected:
{"status":"healthy","version":"1.0.0"} # Test 2: Core API with valid
admission cert header curl -sk https://localhost:443/v1/node/status \ -H
"X-Aethel-Admission-Cert: $(cat /aethel/certs/admission.crt | base64 -w0)" \
-H "X-Aethel-Codex-Version: 1.0.0" | jq . # Expected: {"node_id":"node-
bethel-01","status":"admitted","peers":3} # Test 3: Constitutional gate
rejection (no cert) curl -sk https://localhost:443/v1/node/status # Expected
HTTP 403: Constitutional Gate Violation: admission certificate required #
Test 4: Cruxlux routing audit log – verify requests are being logged tail -
f /aethel/logs/cruxlux-constitutional-audit.log | jq .
```

PHASE 5

VALIDATOR ACTIVATION

This phase activates the node's validator function. The validator participates in constitutional consensus, attesting to state transitions and earning rewards as defined in the L05 Validator Specification.

Step 5.1 — Generate the Validator Key

```
# Generate a dedicated validator key (separate from the node identity key)
openssl genpkey -algorithm ed25519 \ -out /aethel/keys/validator.key
openssl pkey -in /aethel/keys/validator.key \ -pubout -out
/aethel/certs/validator.pub chmod 400 /aethel/keys/validator.key chmod
444 /aethel/certs/validator.pub
```

Step 5.2 — Register the Validator

```
# Submit validator registration to aethel-core curl -X POST
http://localhost:8080/v1/validator/register \ -H "Content-Type:
application/json" \ -H "X-Aethel-Admission-Cert: $(cat
```

```
/aethel/certs/admission.crt | base64 -w0)" \ -H "X-Aethel-Codex-Version:
1.0.0" \ -d "{      \"node_id\": \"$
{AETHEL_NODE_ID}\",      \"validator_public_key\": \"$(cat
/aethel/certs/validator.pub | base64 -w0)\",      \"stake_amount\": $
{VALIDATOR_STAKE_AMOUNT},      \"codex_version\": \"${AETHEL_CODEX_VERSION}\"
}" | jq . # Expected response: # {"validator_id": "val-node-bethel-01",
"status": "registered", "stake": 1000}
```

Step 5.3 — Load the Validator Certificate

```
# After registration, retrieve the validator certificate from the
Constitutional CA curl -s http://localhost:8080/v1/validator/certificate \
-H "X-Aethel-Admission-Cert: $(cat /aethel/certs/admission.crt | base64 -w0)"
\ -H "X-Aethel-Codex-Version: 1.0.0" \ -o /aethel/certs/validator.crt #
Verify the certificate openssl x509 -in /aethel/certs/validator.crt -noout -
subject -issuer -dates # Set permissions chmod 444
/aethel/certs/validator.crt
```

Step 5.4 — Start the Validation Loop

```
# Restart the validator-engine container to load the new certificate docker
compose restart validator-engine # Monitor validator engine startup docker
compose logs -f validator-engine # Expected log output indicating validator
loop started: # {"level":"info","service":"validator-engine","msg":"validator
loop started", # "validator_id":"val-node-bethel-01","slot":1024,"peers":3}
# Confirm the validator is participating in consensus curl -s
http://localhost:9090/health | jq . # Expected:
{"status":"healthy","validator_active":true,"consensus_round":1024}
```

Step 5.5 — Confirming Validator in Governance Dashboard

11. Open the Governance Dashboard at `http://<node-ip>:3000` in a browser
12. Navigate to **Validators > Active Validators**
13. Locate your node by `AETHEL_NODE_ID` (e.g., `node-bethel-01`)
14. Confirm status shows **ACTIVE** with a green indicator
15. Verify the **Attestation Rate** is > 95% after the first 10 consensus rounds
16. Confirm the **Last Attested Slot** is incrementing in real time

◆ Tip

A new validator may show as

PENDING

for up to 5 minutes while it syncs constitutional state and completes its first attestation cycle. This is expected behavior.

PHASE 6

—

HEALTH VERIFICATION

This phase performs a full system health check to confirm that every service is operating correctly and all constitutional invariants are being enforced in the running system.

Step 6.1 — Full System Health Check Commands

```
# 1. Docker Compose service health summary docker compose ps --format "table
{{.Name}}\t{{.Status}}\t{{.Ports}}" # 2. Safety Kernel Daemon health curl -s
http://127.0.0.1:7070/health | jq . # 3. AETHEL Core health and node status
curl -s http://localhost:8080/health | jq . curl -s
http://localhost:8080/v1/node/status \   -H "X-Aethel-Admission-Cert: $(
cat /aethel/certs/admission.crt | base64 -w0)" \   -H "X-Aethel-Codex-
Version: 1.0.0" | jq . # 4. Cruxlux Router health curl -sk
https://localhost:443/cruxlux/health | jq . # 5. Validator Engine health
curl -s http://localhost:9090/health | jq . # 6. Governance Relay health
curl -s http://localhost:9091/health | jq . # 7. Governance Dashboard health
curl -s http://localhost:3000/api/health | jq . # 8. Constitutional
invariant check – all layers curl -s
http://localhost:8080/v1/constitutional/invariants \   -H "X-Aethel-
Admission-Cert: $(cat /aethel/certs/admission.crt | base64 -w0)" \   -H "X-
Aethel-Codex-Version: 1.0.0" | jq .
```

Step 6.2 — Expected Healthy State for Each Service

Service	Health Endpoint	Expected Response Fields	Healthy Value
safety-kernel-daemon	http://127.0.0.1:7070/health	status, invariant_monitor	"healthy", "active"
aethel-core	http://localhost:8080/health	status, peers, admitted	"healthy", ≥ 1 , true
cruxlux-router	https://localhost:443/cruxlux/health	status, gates_active, routes	"healthy", true, ≥ 4
governance-relay	http://localhost:9091/health	status, peer_connections	"healthy", ≥ 1
validator-engine	http://localhost:9090/health	status, validator_active, attestation_rate	"healthy", true, ≥ 0.95
governance-dashboard	http://localhost:3000/api/health	status, data_sync	"healthy", "current"

Step 6.3 — Log Indicators of Healthy vs. Unhealthy State

Log Field / Pattern	Healthy	Unhealthy	Action
"level"	"info"	"error" / "fatal"	Investigate immediately
"invariant_check"	"passed"	"failed" / "violated"	Constitutional halt; see Troubleshooting
"gate_result"	"pass"	"reject"	Review request headers
"consensus_round"	Incrementing	Stale / unchanged	Check peer connectivity
"skd_alert"	Absent	Present	L02 Safety Kernel alert; halt review
"peer_count"	≥ 1	0	Check firewall and peer seeds

Step 6.4 — Governance Dashboard Verification Steps

17. Open `http://<node-ip>:3000` in a browser
18. **Dashboard Overview:** Confirm all six service tiles show green / healthy status
19. **Constitutional Status:** Navigate to **Codex > Invariant Monitor**; all five layers (L01-L05) must show **PASS**
20. **Node Admission:** Navigate to **Network > Admitted Nodes**; confirm your node ID is listed with status **ADMITTED**
21. **Validator Status:** Navigate to **Validators > Active**; confirm your validator shows **ACTIVE** with an attestation rate $\geq 95\%$
22. **Peer Connectivity:** Navigate to **Network > Peers**; confirm at least one peer connection is listed
23. **Constitutional Audit Log:** Navigate to **Audit > Log**; confirm entries are flowing in real time with no gate violation events

PHASE 7

—

OPERATIONAL PROCEDURES

This phase documents the standard operational procedures for the lifecycle management of a running AETHEL Grid node.

7.1 Graceful Restart Procedure

Use this procedure for configuration changes or routine restarts. Services restart in dependency order automatically.


```
# Graceful restart of all services (preserves state volumes) cd /aethel/repo
docker compose restart # Restart a single service (e.g., after config
change) docker compose restart cruxlux-router # Wait for health before
proceeding docker compose ps curl -sk https://localhost:443/cruxlux/health |
jq .status
```

Warning

Restarting

safety-kernel-daemon

will cause all dependent services to also restart due to the Docker Compose

depends_on: condition: service_healthy

chain. Plan for a brief service interruption of 60–90 seconds when restarting the Safety Kernel.

7.2 Node Upgrade Procedure

24. Pull the new image tags from the GitHub Container Registry

25. Update image version tags in `docker-compose.yml`

26. Verify the new release notes for any Codex version changes

27. If the Codex version changes, update `AETHEL_CODEX_VERSION` in `.env`

28. Perform a rolling update: upgrade one service at a time, starting with `safety-kernel-daemon`

29. Verify health after each service upgrade before proceeding to the next

```
# Pull new images docker compose pull # Upgrade safety-kernel-daemon first
docker compose up -d --no-deps safety-kernel-daemon sleep 30 && docker
compose ps safety-kernel-daemon # Upgrade aethel-core next docker compose up
-d --no-deps aethel-core sleep 30 && curl -s http://localhost:8080/health |
jq .status # Continue for remaining services in order: # cruxlux-router →
governance-relay → validator-engine → governance-dashboard
```

7.3 Backup and Restore

Backup — Constitutional State

```
# Stop services before backup to ensure state consistency docker compose stop
validator-engine aethel-core governance-relay BACKUP_DATE=$(date +%Y%m%d-%H
%M%S) tar -czf /backup/aethel-state-${BACKUP_DATE}.tar.gz \
/aethel/state/ \ /aethel/certs/ \ /aethel/config/ # CRITICAL: Back up
validator keys separately to encrypted offline storage tar -czf
/backup/aethel-keys-${BACKUP_DATE}.tar.gz /aethel/keys/ gpg --symmetric --
cipher-algo AES256 /backup/aethel-keys-${BACKUP_DATE}.tar.gz shred -u
/backup/aethel-keys-${BACKUP_DATE}.tar.gz docker compose start governance-
relay aethel-core validator-engine
```

Restore — Constitutional State

```
# Stop all services before restore docker compose down # Restore state from
backup tar -xzf /backup/aethel-state-YYYYMMDD-HHMMSS.tar.gz -C / # Restore
keys (decrypt first) gpg --decrypt /backup/aethel-keys-YYYYMMDD-
HHMMSS.tar.gz.gpg | tar -xzf - -C / # Restart the stack docker compose up -d
```

7.4 Emergency Shutdown Procedure

```
# Immediate halt – stops all containers instantly docker compose down # If
containers are unresponsive, force kill docker compose down --timeout 5 #
Isolate the node from the network (emergency only) sudo iptables -I INPUT 1 -
p tcp --dport 9090 -j DROP sudo iptables -I INPUT 1 -p tcp --dport 9091 -j
DROP # Document the shutdown in the constitutional audit log echo
"{\"event\":\"emergency_shutdown\",\"operator\":\"John\",\"timestamp\":\"$
(date -u +%FT%TZ)\",\"reason\":\"FILL IN\"}" \ >> /aethel/logs/safety-
kernel-audit.log
```

Warning

Emergency shutdown without graceful validator exit will cause the node's validator to be marked offline by peers. The validator will be penalized per the L05 Validator Specification inactivity rules if offline for more than two consecutive consensus epochs. Restore the node and validator as quickly as possible.

7.5 Rolling Back a Deployment

```
# View previous image tags deployed docker image ls | grep iaai33iaai # Edit
docker-compose.yml to reference the previous version tags nano
/aethel/repo/docker-compose.yml # Change image tags back to previous version
(e.g., v0.9.0) # Perform rolling rollback in reverse order docker compose up
-d --no-deps governance-dashboard docker compose up -d --no-deps validator-
```

```
engine docker compose up -d --no-deps governance-relay docker compose up -d
--no-deps cruxlux-router docker compose up -d --no-deps aethel-core docker
compose up -d --no-deps safety-kernel-daemon # Verify full health after
rollback docker compose ps
```

Troubleshooting Guide

This section provides structured diagnosis and resolution procedures for the most common failure modes encountered in AETHEL Grid deployments.

Port Conflict Resolution

Symptom: `docker compose up` fails with *Bind for 0.0.0.0:443 failed: port is already allocated*

```
# Identify what process is holding the port sudo ss -tlnp | grep ':443' sudo
lsof -i :443 # Common culprit: nginx or apache2 on the host sudo systemctl
stop nginx sudo systemctl disable nginx # Verify port is free sudo ss -tlnp
| grep ':443' # Retry compose startup docker compose up -d
```

If the conflicting service cannot be stopped, update the external port binding in `docker-compose.yml` (e.g., change "443:443" to "8443:443") and update the `CRUXLUX_HTTPS_PORT` environment variable accordingly. Update firewall rules to redirect external port 443 to the new host port.

Container Fails to Start

Symptom	Likely Cause	Resolution
safety_kernel_daemon exits immediately	Node key permissions incorrect; key not found	Run <code>chmod 400 /aethel/keys/node.key</code> ; verify path in volume mount
aethel_core exits with CODEX_VERSION_MISMATCH	AETHEL_CODEX_VERSION in .env does not match image	Update AETHEL_CODEX_VERSION in .env to match image-baked version
cruxlux_router exits with TLS error	TLS cert/key path mismatch or expired certificate	Verify cert paths in <code>cruxlux.config.yml</code> ; regenerate if expired
Any service exits with SKD_NOT_READY	Safety Kernel Daemon not yet healthy when service started	Wait for SKD health, then retry: <code>docker compose up -d <service></code>

Symptom	Likely Cause	Resolution
Image pull fails with 401	Not authenticated to GitHub Container Registry	Run <code>docker login ghcr.io -u <github-username> -p <token></code>
Volume mount error: no such file or directory	Host directory not created	Run Phase 1, Step 1.1 directory creation commands

Node Admission Failure (Invariant Violation)

Symptom: Admission status returns `"admission_status": "rejected"` with one or more invariant checks showing `"failed"`

```
# Retrieve the full admission rejection details curl -s
http://localhost:8080/v1/constitutional/admission/status \ -H "X-Aethel-
Node-Id: ${AETHEL_NODE_ID}" | jq . # Check which invariant failed curl -s
http://localhost:8080/v1/constitutional/admission/status \ -H "X-Aethel-
Node-Id: ${AETHEL_NODE_ID}" | jq '.invariant_checks'
```

Failed Invariant	Root Cause	Resolution
L01_MISSION	Invalid or expired Admission Token; genesis hash mismatch	Obtain a new admission token from the Founder-Architect; verify <code>AETHEL_L01_GENESIS_HASH</code>
L02_ISOLATION	Safety Kernel Daemon not running; process isolation violation detected	Ensure <code>safety-kernel-daemon</code> is healthy before submitting proof; do not run as root
L03_STATE_VALIDITY	Corrupted or pre-existing state that doesn't match L03 Formal Model	Clear state directory (<code>/aethel/state/core</code>) and restart from genesis
L04_EXECUTION_HARNESS	Codex version mismatch between node and network	Align <code>AETHEL_CODEX_VERSION</code> in <code>.env</code> with the current network Codex version
L05_VALIDATOR_SPEC	Validator Specification version incompatible	Upgrade <code>validator-engine</code> image to the current network-compatible version

After resolving the root cause, regenerate the admission proof (Phase 3, Step 3.1) and resubmit. A failed admission does not ban the node permanently; it may be retried after correction.

Validator Goes Offline

```
# Check validator engine status docker compose ps validator-engine docker
compose logs --tail=100 validator-engine # Check if validator key
permissions changed ls -la /aethel/keys/validator.key # Must be: -r-----
(mode 0400) # Check validator certificate expiry openssl x509 -in
/aethel/certs/validator.crt -noout -dates # If certificate expired, re-
register validator (Phase 5) # Check peer connectivity from validator docker
exec validator_engine curl -s http://governance_relay:9091/health | jq . #
Restart validator engine after resolving root cause docker compose restart
validator-engine docker compose logs -f validator-engine
```

Cruxlux Routing Errors

Symptom: API requests return 503 Service Unavailable or 502 Bad Gateway

```
# Check Cruxlux router logs docker compose logs --tail=50 cruxlux-router |
grep -E "error|warn|upstream" # Verify upstream connectivity from Cruxlux
container docker exec cruxlux_router curl -s http://aethel_core:8080/health |
jq . # If upstream is unreachable, check Docker network docker network
inspect aethel-repo_aethel-net # Validate cruxlux.config.yml syntax docker
exec cruxlux_router cruxlux-cli config validate \ --config
/etc/cruxlux/cruxlux.config.yml # Reload config after fixing docker kill --
signal=SIGHUP cruxlux_router
```

Symptom: All requests returning 403 Constitutional Gate Violation

```
# Verify admission cert is valid openssl x509 -in /aethel/certs/admission.crt
-noout -dates # Test with explicit cert header curl -vsk
https://localhost:443/v1/node/status \ -H "X-Aethel-Admission-Cert: $
(cat /aethel/certs/admission.crt | base64 -w0)" \ -H "X-Aethel-Codex-
Version: 1.0.0" # If cert is expired, re-run node admission (Phase 3)
```

Safety Kernel Daemon Alerts

Symptom: SKD alert in logs: "skd_alert": "invariant_violation"

```
# Review the Safety Kernel audit log tail -100 /aethel/logs/safety-kernel-
audit.log | jq \ 'select(.event_type == "invariant_violation")' # Check
which invariant was violated and on which service tail -100
/aethel/logs/safety-kernel-audit.log | jq \ '{event: .event_type,
layer: .codex_layer, service: .service, detail: .detail}' # If SKD triggered
a halt, services may be stopped docker compose ps # DO NOT restart blindly –
understand the invariant violation first # Review the violation detail and
remediate before restarting # Once remediated, restart SKD and the stack
docker compose down docker compose up -d
```

Critical Warning

A Safety Kernel Daemon invariant halt is a constitutional protection event — not a bug. Do not attempt to disable the SKD or bypass its halt mechanism. Investigate

the invariant violation thoroughly. If you cannot resolve it, contact the Founder-Architect before bringing the node back online.

Security Hardening

Firewall Configuration

Apply the following iptables rules to enforce the principle of least privilege at the network boundary. These rules restrict external access to only the ports required for public operation.

```
# Allow established connections sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -i lo -j ACCEPT # Allow SSH (adjust port as needed)
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT # Allow Cruxlux HTTPS (public)
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT # Allow Validator consensus RPC (peer nodes only – restrict to known peers)
sudo iptables -A INPUT -p tcp --dport 9090 -s <PEER1_IP> -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 9090 -s <PEER2_IP> -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 9090 -j DROP # Allow Governance Relay (peer nodes only)
sudo iptables -A INPUT -p tcp --dport 9091 -s <PEER1_IP> -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 9091 -j DROP # Block port 8080 (Core API – internal only)
sudo iptables -A INPUT -p tcp --dport 8080 -j DROP # Block port 3000 (Governance Dashboard – restrict to operator IPs)
sudo iptables -A INPUT -p tcp --dport 3000 -s <OPERATOR_IP> -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 3000 -j DROP # Block port 7070 (SKD – localhost only, Docker handles this)
sudo iptables -A INPUT -p tcp --dport 7070 ! -i lo -j DROP # Default deny inbound
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP # Save rules
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Key Rotation Schedule

Key / Certificate	Rotation Frequency	Procedure	Notes
TLS Certificate (Cruxlux)	90 days (Let's Encrypt) or annually (internal CA)	Generate new cert; reload Cruxlux (SIGHUP)	Automate with Certbot for Let's Encrypt
Validator Certificate	Annually or upon compromise	Re-register validator (Phase 5)	Requires brief validator downtime
Node Identity Key	Only upon	Full re-admission	Rotating the node

Key / Certificate	Rotation Frequency	Procedure	Notes
	compromise	ceremony required (Phase 3)	key changes the Node ID
Admission Certificate	Tied to Node Identity Key lifecycle	Re-run Phase 3 after key rotation	Issued by Constitutional CA at admission

TLS Certificate Renewal

```
# Automated renewal with Certbot (Let's Encrypt) – run as cron job
certbot renew --standalone --pre-hook "docker compose stop cruxlux-router" \
  --post-hook "cp /etc/letsencrypt/live/<domain>/fullchain.pem /aethel/certs/cruxlux.crt && \
    cp /etc/letsencrypt/live/<domain>/privkey.pem /aethel/certs/cruxlux.key && \
    docker compose start cruxlux-router" # Crontab entry (renew daily at 03:00,
Certbot only acts when expiry is <30 days) # 0 3 * * * /usr/bin/certbot renew
--quiet ... # Manual certificate expiry check
openssl x509 -in /aethel/certs/cruxlux.crt -noout -dates echo | openssl s_client -connect
localhost:443 2>/dev/null | \ openssl x509 -noout -dates
```

Audit Log Configuration

```
# Verify audit log is being written
tail -f /aethel/logs/safety-kernel-audit.log # Set up log rotation (logrotate)
sudo nano /etc/logrotate.d/aethel
# /etc/logrotate.d/aethel
/aethel/logs/*.log {
    daily rotate 90
    compress delaycompress missingok notifempty
    sharedscripts
    postrotate
        docker kill --signal=SIGUSR1 safety_kernel_daemon
    2>/dev/null || true
    endscript
}
# Ship audit logs to central SIEM (example: rsyslog forward) # Add to
/etc/rsyslog.d/aethel.conf: # module(load="imfile") # input(type="imfile"
File="/aethel/logs/safety-kernel-audit.log" # Tag="aethel-skd-audit"
Severity="info") # *.* @@<SIEM_HOST>:514
```

APPENDIX A

Full Port Reference Table

Port	Protocol	Service	Container Name	Binding	Access Control	Description
443	TCP/HTTPS	cruxlux-router	cruxlux_router	0.0.0.0:443	Public (TLS + constitutional gates)	Primary HTTPS API gateway. All

Port	Protocol	Service	Container Name	Binding	Access Control	Description
						external client traffic enters here.
3000	TCP/HTTP	governance-dashboard	governance_dashboard	0.0.0.0:3000	Restrict to operator IPs via firewall	Read-only governance and validator monitoring dashboard.
6060	TCP/HTTP	cruxlux-router	cruxlux_router	0.0.0.0:6060	Internal network only; firewall externally	Cruxlux internal routing bus for inter-service communication.
7070	TCP	safety-kernel-daemon	safety_kernel_daemon	127.0.0.1:7070	Localhost only — never expose externally	Safety Kernel Daemon control socket. Health and invariant status.
8080	TCP/HTTP	aethel-core	aethel_core	0.0.0.0:8080	Internal network only; block externally	AETHEL Core REST API. Used by Cruxlux and internal tooling.
8443	TCP/HTTPS	aethel-core	aethel_core	0.0.0.0:8443	mTLS — peer nodes only	AETHEL Core peer protocol with mutual TLS. Used for peer state sync.
9090	TCP/gRPC	validator-engine	validator_engine	0.0.0.0:9090	Restrict to known peer node	Validator consensus RPC. Used

Port	Protocol	Service	Container Name	Binding	Access Control	Description
					IPs	for attestation and block validation.
9091	TCP	governance-relay	governance-relay	0.0.0.0:9091	Restrict to known peer node IPs	Governance event mesh. Propagates votes and constitutional events.

APPENDIX B

Environment Variable Reference

Variable	Required	Default	Description
AETHEL_NODE_ID	Yes	—	Unique node identifier. Derived from node public key fingerprint.
AETHEL_NODE_REGION	Yes	—	Geographic region of the node (e.g., us-central).
AETHEL_NETWORK_ID	Yes	—	Network identifier (e.g., aethel-mainnet-v1). Must match peers.
AETHEL_NODE_PUBLIC_IP	Yes	—	Public IPv4 address of the node. Used in TLS SAN and peer discovery.
AETHEL_CODEX_VERSION	Yes	—	Codex version this node implements (e.g., 1.0.0). Must match network.
AETHEL_L01_GENESIS_HASH	Yes	—	Hash of the L01

Variable	Required	Default	Description
HASH			Root Constitution genesis block. Provided by Founder-Architect.
AETHEL_ADMISSION_TO_KEN	Yes	—	Single-use constitutional admission token. Issued by Root Constitution authority.
AETHEL_STATE_DIR	Yes	/aethel/state	Base path for constitutional state storage on the host.
AETHEL_KEY_DIR	Yes	/aethel/keys	Path to cryptographic key files. Must be mode 750 or stricter.
AETHEL_CERT_DIR	Yes	/aethel/certs	Path to certificate files (TLS, admission, validator).
CRUXLUX_HTTPS_PORT	Yes	443	External HTTPS port for the Cruxlux API gateway.
CRUXLUX_INTERNAL_PORT	Yes	6060	Cruxlux internal routing bus port.
CRUXLUX_TLS_CERT	Yes	—	Absolute path to the Cruxlux TLS certificate file.
CRUXLUX_TLS_KEY	Yes	—	Absolute path to the Cruxlux TLS private key file.
CRUXLUX_CONSTITUTIONAL_GATE_MODE	No	strict	Gate enforcement mode: strict, permissive, or audit-only.
CORE_HTTP_PORT	No	8080	AETHEL Core REST API port.
CORE_MTLS_PORT	No	8443	AETHEL Core mTLS peer protocol port.
CORE_PEER_SEEDS	Yes	—	Comma-separated list of peer node addresses for initial peer discovery.
VALIDATOR_RPC_PORT	No	9090	Validator Engine

Variable	Required	Default	Description
			gRPC consensus port.
VALIDATOR_CERT	Yes	—	Path to validator certificate issued by Constitutional CA.
VALIDATOR_KEY	Yes	—	Path to validator private key. Mode must be 0400.
VALIDATOR_STAKE_AMOUNT	Yes	1000	Stake amount for validator registration per L05 specification.
GOVERNANCE_RELAY_PORT	No	9091	Governance Relay peer mesh port.
GOVERNANCE_DASHBOARD_PORT	No	3000	Governance Dashboard HTTP UI port.
SKD_CONTROL_PORT	No	7070	Safety Kernel Daemon control socket port. Bound to localhost only.
SKD_INVARIANT_HALT_MODE	No	graceful	SKD halt mode on invariant violation: graceful or immediate.
SKD_AUDIT_LOG	No	/aethel/logs/safety-kernel-audit.log	Path for Safety Kernel Daemon constitutional audit log.
LOG_LEVEL	No	info	Log verbosity: debug, info, warn, error.
LOG_FORMAT	No	json	Log output format: json or text. JSON required for audit compliance.

Constitutional Admission Proof Schema

The Node Admission Proof is a signed JSON document submitted to the Constitutional Kernel during Phase 3. All fields are required unless marked optional.

Field	Type	Required	Description
proof_version	string	Yes	Schema version of this proof document. Must be "1.0".
node_id	string	Yes	Unique node identifier matching AETHEL_NODE_ID.
public_key_fingerprint	string	Yes	SHA-256 fingerprint of the node Ed25519 public key. Format: sha256:<hex>.
public_key_pem	string	Yes	PEM-encoded Ed25519 public key of the node.
codex_version	string	Yes	Codex version the node implements (e.g., "1.0.0"). Must match network.
l01_genesis_hash	string	Yes	Hash of the L01 Root Constitution genesis block. Prefixed 0x.
admission_token	string	Yes	Single-use admission token issued by Root Constitution authority.
network_id	string	Yes	Network identifier the node is joining (e.g., "aethel-mainnet-v1").
node_public_ip	string	Yes	Public IPv4 address of the node. Used for peer discovery registration.

Field	Type	Required	Description
invariants_asserted	array of string	Yes	List of Codex invariant identifiers the node asserts compliance with.
software_version	string	Yes	AETHEL Grid software version running on the node (e.g., "1.0.0").
skd_version	string	Yes	Safety Kernel Daemon version (e.g., "1.0.0"). Must be $\geq$ minimum required.
timestamp	string (ISO 8601)	Yes	Proof generation timestamp in UTC. Proof is invalid if age > 10 minutes at submission.
operator_name	string	No	Optional: Human-readable name of the node operator for governance records.
operator_contact	string	No	Optional: Contact email or identifier for the node operator.
signature	string (base64)	Yes	Ed25519 signature over all other fields, serialized as canonical JSON and encoded as base64. Must verify against public_key_pem.

Full Proof Example

```
{  "proof_version": "1.0",  "node_id": "node-bethel-01",
"public_key_fingerprint": "sha256:a3f2c1d8e4b7...",  "public_key_pem":
"-----BEGIN PUBLIC KEY-----\nMCow...AQAB\n-----END PUBLIC KEY-----",
"codex_version": "1.0.0",  "l01_genesis_hash": "0xdeadbeef1234...",
"admission_token": "AETHEL-TOKEN-XXXXXXXXXXXXXXXXXX",  "network_id": "aethel-
mainnet-v1",  "node_public_ip": "192.0.2.100",  "invariants_asserted": [
"L01.MISSION",  "L02.ISOLATION",  "L03.STATE_VALIDITY",
"L04.EXECUTION_HARNESS",  "L05.VALIDATOR_SPEC"  ],  "software_version":
"1.0.0",  "skd_version": "1.0.0",  "timestamp": "2026-07-12T20:46:00Z",
"operator_name": "John David Taylor Preston",  "operator_contact":
"founder@aethel.network",  "signature": "base64-encoded-ed25519-signature-
over-all-above-fields..." }
```

Glossary of AETHEL Grid Terms

Term	Definition
AETHEL Grid	A constitutional sovereign compute protocol governed by the five-layer Codex of invariant constraints. Designed to support a world where humans and AI thrive together.
Codex	The five-layer constitutional specification (L01-L05) that governs all AETHEL Grid behavior. The Codex is the supreme law of the network; no configuration or code may override it.
Constitutional Invariant	A formally specified property that must hold true at all times in a running AETHEL Grid system. Violation of an invariant triggers the Safety Kernel Daemon halt mechanism.
Constitutional Gate	A declarative access control rule in the Cruxlux router that evaluates incoming API requests against constitutional requirements before forwarding them upstream.
Constitutional Geometry	The principle that the topology and routing structure of the Cruxlux API layer must itself satisfy L01 constraints. The "shape" of how requests flow through the system is constitutionally governed.
Cruxlux	The AETHEL Grid sub-protocol responsible for API routing and constitutional geometry enforcement. All external and inter-service traffic passes through Cruxlux.
Node Admission	The formal ceremony by which a new node generates a proof of invariant compliance and is recognized as a participant in the AETHEL Grid network. Admission is constitutional, not merely technical.
Node Admission Proof	A signed JSON document asserting the node's identity, software version, constitutional alignment, and invariant compliance. Required for network admission.

Term	Definition
Node ID	A unique identifier for a node, derived from the SHA-256 fingerprint of the node's Ed25519 public key. Immutable for the lifetime of the node key pair.
Safety Kernel Daemon (SKD)	The L02 runtime invariant monitor. Runs first in the service stack and monitors all other services. May halt services upon constitutional invariant violation.
L01 — Root Constitution	The supreme layer of the Codex. Defines the AETHEL Grid mission, participant rights, and constitutional authority. Cannot be overridden by any lower layer.
L02 — Safety Kernel	The runtime enforcement layer. Implements the Safety Kernel Daemon, process isolation requirements, and invariant halt procedures.
L03 — Formal Model	The mathematical specification of the constitutional state machine. Defines valid state transitions and proof obligations that all nodes must satisfy.
L04 — Execution Harness	The trusted execution environment for constitutional compute. All workloads run within invariant-checked execution contexts defined by L04.
L05 — Validator Specification	The protocol specification governing validator node behavior: election, attestation, slashing conditions, and reward rules.
Validator	A node that participates in constitutional consensus by attesting to state transitions, validating blocks, and earning rewards as defined in the L05 Validator Specification.
Validator Certificate	An X.509 certificate issued by the Constitutional CA to a registered validator node. Required to participate in validator consensus RPC.
Governance Dashboard	A read-only web UI displaying real-time validator status, node health, constitutional invariant states, and audit event log.
Governance Relay	The P2P governance message bus that propagates validator state, constitutional votes, and governance events across all network nodes.
Genesis Hash (L01)	The cryptographic hash of the L01 Root Constitution genesis block. All nodes on the same network must share an identical genesis hash.

Term	Definition
Invariant Halt	An emergency stop triggered by the Safety Kernel Daemon when a constitutional invariant violation is detected. All affected services are halted to preserve network integrity.
Constitutional Audit Log	An append-only structured log of all constitutional events: gate evaluations, invariant checks, admission ceremonies, and SKD alerts. Required for compliance.
Attestation	A validator's cryptographic assertion that a proposed state transition is valid per the L03 Formal Model. Attestations from a quorum of validators constitute constitutional consensus.
Slashing	A penalty applied to a validator per L05 rules when it provably violates the Validator Specification (e.g., double-attestation or prolonged downtime).
aethel-net	The isolated Docker bridge network on which all AETHEL Grid services communicate. Separate from the host network for security isolation.
Founder-Architect	John David Taylor Preston. The constitutional authority for the Root Constitution (L01). Issues admission tokens, genesis hashes, and acts as the Root CA for the AETHEL Grid network.

Document prepared and authorized by:

John David Taylor Preston

Founder-Architect, AETHEL Grid — iAAi33iAAi

July 12, 2026

AETHEL Grid Full Deployment Runbook v1.0 | Document Class:
 Operations/Infrastructure | Issued: 2026-07-12T20:46:00-05:00
 GitHub: github.com/iAAi33iAAi | Mission: Design a world where
 humans and AI thrive together.